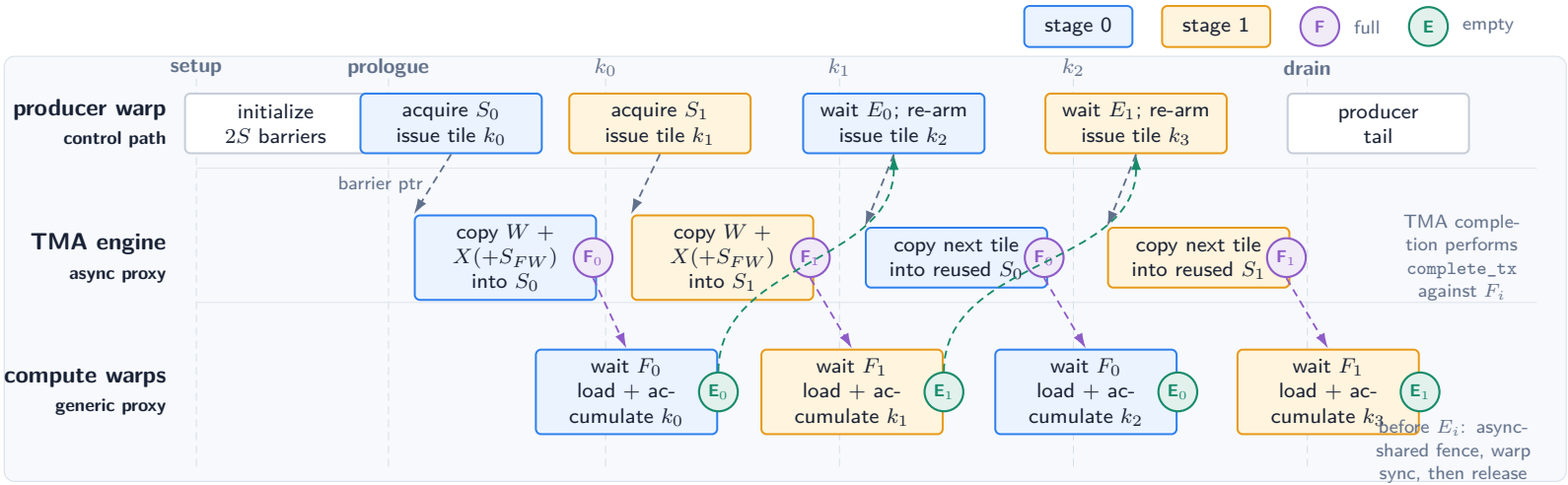
Blockscaled TMA pipeline: how the mbarriers hand off each stage

Two-stage main TMA path from `BlockscaledTmaGemv`; optional input-scale pipeline omitted; not cycle-accurate



One circular stage owns two 64-bit mbarriers

F_i tracks "data has arrived"; E_i tracks "consumers are done." The kernel allocates `num_stages * 2 Int64` entries.

CuTe pipeline API mapping

`acquire_and_advance()` waits E_i and arms F_i with expected bytes. Each `cute.copy(..., tma_bar_ptr=...)` reports completion to F_i; TMA `commit()` is therefore a no-op.

Index/phase prevents stale completion

For $S = 2$: producer $(0,1) \to (1,1) \to (0,0)$; consumer $(0,0) \to (1,0) \to (0,1)$. The phase flips whenever the ring wraps.

Expected transaction bytes in this kernel: `tx_count` $=$ $(\texttt{block\_n} + m)\,\texttt{tile\_k\_u32} \times 4 +$ optional staged weight-scale bytes.

Source anchors: barrier allocation and pipeline creation in `transformer_nuggets/cute/blockscaled_tma.py`; producer load in `tma_producer_load_stage`; consumer handoff in `compute_warp_consume_stage`.